



Group Members

Student 1: Gaurav Acharya — [MT26MCS024]

Student 2: Sandeep — [MT26MCS032]

Guide Name

Dr. A. S. Mokhade sir

Practical Title:

CSP529 Technical Writing & Publishing:
Practical 02: From Engineering Thinking to Scientific Writing

Problem Statement:

Concept: Operating System Process

Stage 1

Q. How many different ways can one technical idea be communicated?

There are multiple ways a technical idea can be communicated to technical and non-technical staff using the following formats:

1. Written Documents:

Words on paper or screens explain how things work in detail.

- **Specifications:** Lists of exact rules, features, and requirements.
- **White papers:** In-depth reports that explain a problem and a new technical fix.
- **User manuals:** Guides that teach people how to use the final product.

2. Visual Diagrams:

Pictures show how parts connect without using long blocks of text.

- **Flowcharts:** Maps that show the steps in a process.
- **Schematics:** Blueprints or maps of electrical and mechanical systems.
- **Wireframes:** Basic sketches of a software screen or layout.

3. Oral Presentations

Speaking helps clear up confusion and lets people ask questions right away.

- **Meetings:** Live talks with a small team.
- **Pitch decks:** Short slideshows with a spoken talk for investors or bosses.
- **Video demos:** Recorded clips showing the idea in action.

4. Code and Data

Computers and engineers understand raw logic and numbers best.

- **Source code:** Real programming instructions that build a feature.
- **Pseudo-code:** Fake, simple code used to explain the logic to humans.
- **APIs:** Rules that show how two computer systems communicate.

5. Prototypes and Models

Touching or testing a sample makes the idea feel real.

- **Mockups:** Fake, non-working models of a physical item or app.
- **Clickable demos:** Early software you can click through.
- **3D prints:** Plastic or metal shapes made by a machine to test fit.

Stage 2

Paragraph Representation of Idea:

An operating system (OS) acts as the foundational software layer that manages and controls all hardware and software components within a computer. Its startup sequence begins with a small program called a bootloader, which initializes the hardware and loads the core of the OS—the kernel—into main memory. Once activated, the kernel assumes full control of the system, efficiently allocating CPU time, managing memory processes, coordinating input/output devices, and providing a stable, secure environment where user applications can seamlessly execute.

Figure Representation of Idea:

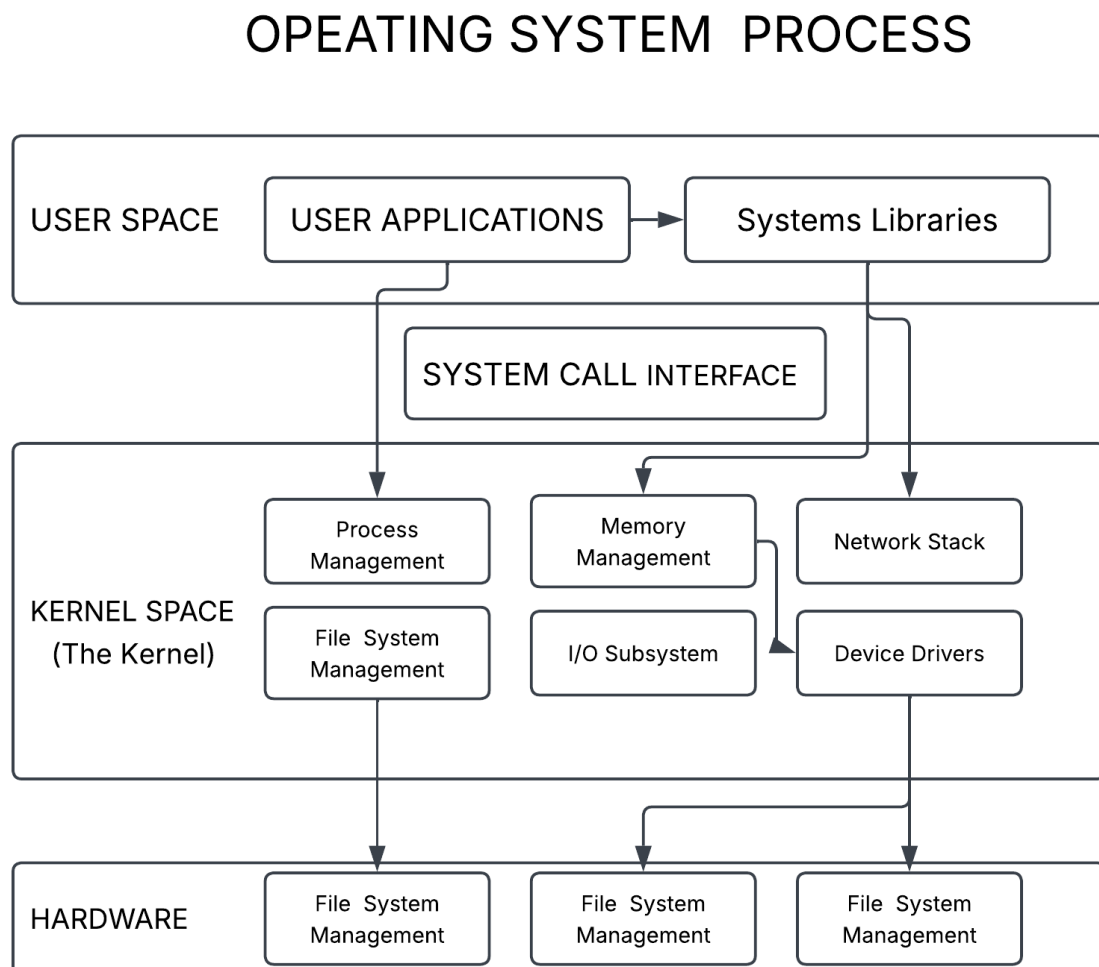


Table Representation of Idea:

System Component	Primary Function	Execution Phase
Firmware (BIOS/UEFI)	Performs hardware checks (POST) and finds the boot device	Immediate Power-On
Bootloader	Loads the kernel into main memory	Early Boot Sequence
Kernel	Manages CPU, RAM, and hardware resource allocation	Continuous Operation
Shell / User Space	Provides the interface for user applications to run	Post-Boot / Active

Equation Representation of Idea:

$$S_{t+1} = k \left(\frac{S_t, \sum_{i=1}^h I_i(t) + \sum_{j=1}^s C_j(t)}{\quad} \right) - \Delta_{overhead}$$

Equation Breakdown

- S_{t+1} (**Next System State**): The resulting state of the computer (e.g., updated memory allocation, next thread in the CPU, written data on the disk).
- K (**The Kernel Function**): The core operating system logic that processes inputs, enforces security, and allocates hardware resources.
- S_t (**Current System State**): The exact configuration of the CPU registers, RAM, and I/O devices at the current moment in time.
- $\sum I_{i(t)}$ (**Hardware Interrupts**): The sum of all hardware signals happening at time t (e.g., a mouse click, a network packet arriving, a disk finishing a read).
- $\sum C_{j(t)}$ (**System Calls**): The sum of all software requests from user applications at time t (e.g., a browser asking for memory or attempting to write a file).
- $\Delta_{\{overhead\}}$ (**Context Switching Overhead**): The inevitable computational time and resources the OS consumes just to manage these transitions, representing the "tax" of running an OS.

Algorithmic Representation of Idea:

START OS_Boot_Sequence

Initialize CPU and apply power

Run Power-On Self Test (POST)

IF hardware fails POST THEN

HALT and display error code

ELSE

Identify primary boot device (SSD/HDD/Network)

Load Bootloader from EFI System Partition / MBR

Execute Bootloader

Bootloader decompresses and loads Kernel into RAM

Pass system control to Kernel

Kernel initializes device drivers and memory structures

Kernel mounts the root file system

Kernel starts background system services (daemons)

Launch Graphical User Interface (GUI) or Command Line Interface (CLI)

END IF

END OS_Boot_Sequence

Bullet Point Representation of Idea:

The core responsibilities of an Operating System include:

- **Process Management:** Creating, suspending, scheduling, and terminating software processes.
- **Memory Management:** Tracking available RAM and securely allocating it to active applications.
- **File System Management:** Organizing directories, writing data to storage, and enforcing file permissions.
- **Device Management:** Using drivers to facilitate seamless communication between the software and physical peripherals.

Numbered List Representation of Idea:

1. The physical machine is powered on, sending a wake signal to the CPU.
2. The motherboard firmware (BIOS/UEFI) executes a hardware diagnostic check.
3. The firmware locates the bootable storage drive.
4. The system executes the bootloader (such as GRUB or Windows Boot Manager).
5. The bootloader transfers the core OS kernel into the system's memory.
6. Kernel takes over hardware management, starts services, and presents the login screen to user.

Stage 3

```
\documentclass{article}

\usepackage{graphicx}
\usepackage{algorithm}
\usepackage{algpseudocode}

\begin{document}

\section{Introduction}

This is an example of a section in LaTeX.

\subsection{Background}

This is an example of a subsection.

\section{Table}

\begin{table}[h]
\centering
\begin{tabular}{|c|c|}
\hline
Name & Marks \\
\hline
Sandeep & 90 \\
Rahul & 85 \\
\hline
\end{tabular}
\caption{Student Marks}
\label{tab:marks}
\end{table}

\section{Equation}

\begin{equation}
y = mx + c
\label{eq:linear}
\end{equation}

\section{Figure}

\begin{figure}[h]
\centering
\includegraphics[width=0.4\textwidth]{example.png}
\caption{Example Figure}

\label{fig:example}
\end{figure}

\section{Algorithm}

\begin{algorithm}
\caption{Linear Search}
\begin{algorithmic}
\State $i \leftarrow 1$
\While{$i \leq n$}
  \If{$A[i] = x$}
    \State \Return $i$
  \EndIf
  \State $i \leftarrow i + 1$
\EndWhile
\State \Return -1
\end{algorithmic}
\end{algorithm}

\section{Itemize}

\begin{itemize}
\item Python
\item C++
\item Java
\end{itemize}

\section{Enumerate}

\begin{enumerate}
\item Input data
\item Process data
\item Generate output
\end{enumerate}

\section{Cross-reference}

Equation \ref{eq:linear} shows a linear equation.

Table \ref{tab:marks} contains student marks.

Figure \ref{fig:example} shows an example figure.

\end{document}
```

1 Introduction

This is an example of a section in LaTeX.

1.1 Background

This is an example of a subsection.

2 Table

Name	Marks
Sandeep	90
Rahul	85

Table 1: Student Marks

3 Equation

$$y = mx + c \tag{1}$$

4 Figure

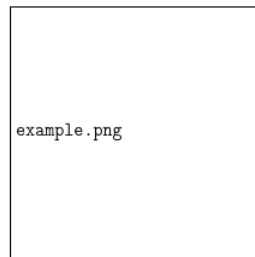


Figure 1: Example Figure

Algorithm 1 Linear Search

```
i ← 1
while i ≤ n do
  if A[i] = x then
    return i
  end if
  i ← i + 1
end while
return -1
```

5 Algorithm

6 Itemize

- Python
- C++
- Java

7 Enumerate

1. Input data
2. Process data
3. Generate output

8 Cross-reference

Equation 1 shows a linear equation.
Table 1 contains student marks.
Figure 1 shows an example figure.

Stage 4

Evaluation by the Receiving Group

Siddharth Gupta M3 , Manav Kaushal M4 , Gopireddy Pritham Reddy M29

Would it be possible to understand the concept without the explanations of the author?

Yes, mostly. With the help of different representations, enough knowledge has been obtained about the working principle of RSA, including its objective, key points, mathematics, and procedures.

However, one thing we observed was that there could have been a mention of RSA security due to its complexity of factorization in its algorithm. This would have made its technical explanation more complete.

What is missing from the representations?

There has not been found any major representation missing in the paper. With the help of the present paragraph, table, equation, algorithm, and list, we can obtain conceptual, mathematical, comparative and procedural understanding.

Table 3. Marks rewarded by the receiving group

Criterion	Marks	Comments
Correctness	2/2	Technical concepts and equations are correct.
Clarity	2/2	Representations are easy to follow.
Completeness	1/2	Security basis could have been stated more explicitly in the algorithm/explanation.
Representation choice	2/2	Each representation serves a distinct purpose
LaTeX quality	2/2	Structure and mathematical representation are appropriately implemented
Total	9/10	Minor technical omission Only

Stage 5

E1: Which representation would you modify? Why?

I would mainly modify the **Written Document and Visual Diagram** according to the audience.

- **Mechanical Engineer:** Use technical language and mechanical-system examples in diagrams.
- **Civil Engineer:** Use civil engineering examples and relevant schematics.
- **Dean:** Keep the written document concise, formal, and focused on important findings.
- **Industry Engineer:** Emphasize practical applications, implementation, and real-world results.
- **School Student:** Use simple language, easy examples, and clear visual diagrams while reducing technical terms.

Why? Because each audience has a different level of technical knowledge and different requirements for understanding the same idea.

E2: Do you discover that audience changes writing?

Yes. The audience changes both the **writing and representation**. The same idea may need a detailed technical document for an engineer, a concise formal document for a Dean, and simple text with diagrams for a school student.

Final Reflection

G) Every group discusses and writes:

1. Which representation communicated my idea best?

The **visual diagram** communicated my idea best because flowcharts and diagrams can show the relationship between different parts and steps clearly without using long blocks of text.

2. Which representation was most difficult?

Code and data was the most difficult because it requires precise syntax and technical understanding, and even a small mistake can change the meaning.

3. If I removed one representation, which one would reduce understanding the most?

Removing the **visual diagram** would reduce understanding the most because diagrams provide a quick visual explanation of how different components or steps are connected.

4. If AI generated your report instantly, what learning opportunities would you have lost by preparing it yourself?

If AI generated the report instantly, I would have lost the opportunity to **understand the concept deeply, decide which representation communicates the idea best, organize the information, and learn how to explain the same idea in different forms**. Preparing it myself helped me develop communication and technical thinking skills.